CS3002E Compiler Design

Monsoon 2025

Lecture #31

Introduction to Code Optimization

Department of Computer Science & Engineering
NIT Calicut

Code Improving Transformations

- Machine Independent Optimizations
  - Code Improving Transformations done in the intermediate code to generate better machine code (that runs faster / or takes less space)
- Machine Dependent Optimizations - transformations done in the target code

Code Improving Transformations

- Requires analysis of the code to identify possible improvements.
- Transformation should not change the meaning of the program (should be *semantics-preserving*)

Local Vs. Global Optimization

- Local Optimizations - restricted to portion of code known as basic blocks (a sequence of instructions such that flow of control always enters the first instruction and exits only after executing the last instruction).
- Global Optimization - extend beyond basic blocks, but confined to an individual procedure - uses *data flow analysis* to retrieve the required information. [1]

[1] optimizations that extend beyond procedure boundaries are known as interprocedural optimizations

Exercise

Write the 3-address code generated for $x = (a + b) * (a + b)$ and identify possible code improvements.

Array References: 3-address instructions

Indexed copy instructions of the form:

$x = y[i]$ sets x to the value in the location i memory units beyond location y

$x[i] = y$ copies the contents of y to the location i units beyond x

Array References: 3-address code

Assuming one element requires 4 bytes, the expression $a[i]$ is translated to:

$t_1 = i * 4$
$t_2 = a[t_1]$

Write the 3-address code generated for $b = a[i]$

Array References: 3-address code

$b = a[i]$ translated to:

$t_1 = i * 4$
$t_2 = a[t_1]$
$b = t_2$

Write the 3-address code generated for $x = a[i] + b[i]$

Array References: 3-address code

$x = a[i] + b[i]$ translated to:

$t_1 = i * 4$
$t_2 = a[t_1]$
$t_3 = i * 4$
$t_4 = b[t_3]$
$t_5 = t_2 + t_4$
$x = t_5$

Is there a *redundant* computation?

Redundant Expression: example

$x = a[i] + b[i]$ translated to:

$t_1 = i * 4$
$t_2 = a[t_1]$
$t_3 = i * 4$
$t_4 = b[t_3]$
$t_5 = t_2 + t_4$
$x = t_5$

Redundant Expression: computation of $i * 4$ in the third instruction is redundant. Value is already computed in the first instruction.

How to eliminate the redundancy?

Common Subexpression Elimination (CSE)

Generated Code	After CSE
$t_1 = i * 4$	$t_1 = i * 4$
$t_2 = a[t_1]$	$t_2 = a[t_1]$
$t_3 = i * 4$	$t_3 = t_1$
$t_4 = b[t_3]$	$t_4 = b[t_3]$
$t_5 = t_2 + t_4$	$t_5 = t_2 + t_4$
$x = t_5$	$x = t_5$

Note: The above is an instance of *Local CSE*.
Global CSE will be discussed later.

Further code Improvements?

Copy Propagation

$t_1 = i * 4$
$t_2 = a[t_1]$
$t_3 = t_1$
$t_4 = b[t_3]$
$t_5 = t_2 + t_4$
$x = t_5$

After the third instruction, t_3 has the same value as t_1 and hence uses of t_3 can be replaced by t_1.

Copy Propagation

3-address code	After Copy Propagation
$t_1 = i * 4$	$t_1 = i * 4$
$t_2 = a[t_1]$	$t_2 = a[t_1]$
$t_3 = t_1$	$t_3 = t_1$
$t_4 = b[t_3]$	$t_4 = b[t_1]$
$t_5 = t_2 + t_4$	$t_5 = t_2 + t_4$
$x = t_5$	$x = t_5$

Further Improvements?

3-address code	After Copy Propagation
$t_1 = i * 4$	$t_1 = i * 4$
$t_2 = a[t_1]$	$t_2 = a[t_1]$
$t_3 = t_1$	$t_3 = t_1$
$t_4 = b[t_3]$	$t_4 = b[t_1]$
$t_5 = t_2 + t_4$	$t_5 = t_2 + t_4$
$x = t_5$	$x = t_5$

Further Improvements: If the value of t_3 is not used again, the instruction $t_3 = t_1$ can be removed.

Dead Code Elimination

3-address code	After Dead Code Elimination
$t_1 = i * 4$	$t_1 = i * 4$
$t_2 = a[t_1]$	$t_2 = a[t_1]$
$t_3 = t_1$	$t_4 = b[t_1]$
$t_4 = b[t_3]$	$t_5 = t_2 + t_4$
$t_5 = t_2 + t_4$	$x = t_5$
$x = t_5$	

The value of t_3 is not used again and hence the instruction $t_3 = t_1$ is dead code which is removed.

Simple Transformations

- Use of algebraic identities

  replace $x + 0$ by x

  replace $x * 1$ by x

- Strength Reduction: Replacing an operation by a less expensive one

  replacing multiplication by shift operation

Constant Propagation

Source code:

```
i = 5
x = sum + i * 10
```

3-address code [2]	After Constant Propagation
$i = 5$	$i = 5$
$t_1 = i * 10$	$t_1 = 5 * 10$
$t_2 = sum + t_1$	$t_2 = sum + t_1$
$x = t_2$	$x = t_2$

replaces use of i by 5 - eliminates a memory load

[2] for simplicity, labels generated in translation of sequence are not shown

Constant Folding

Replaces an expression that is known to evaluate to a constant[3], by its value.

3-address code	After Constant Folding
$i = 5$	$i = 5$
$t_1 = 5 * 10$	$t_1 = 50$
$t_2 = sum + t_1$	$t_2 = sum + t_1$
$x = t_2$	$x = t_2$

Replaces `5 * 10` by its value `50`.
Note: if i is not used again, $i = 5$ becomes dead code.

[3] should have the same constant value irrespective of the target machine

Exercise

Write the 3-address code generated for the following sequence of statements and identify common subexpressions, if any, in the generated code:

```
x = a[i]
i = i+1
y = a[i]
```

References

Major Reference:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU Chapter 9.